THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY
Department of Computer Science & Engineering · Patiala

UCS310 · DATABASE MANAGEMENT SYSTEMS · EVEN SEM 2025–26

FINAL PROJECT REPORT

ChainGuard

A Tamper-Evident Database System for Digital Evidence Chain-of-Custody Management

ROLE	NAME	ROLL NUMBER
Team Leader	Arun A K	1024170121
Member	Vansh Kapil	1024170118
Member	Tikshan Garg	1024170105

SUPERVISOR	Ms. Prachi Semwal · Department of CSE, TIET
SUBMISSION DATE	8 May 2026
REPOSITORY	`github.com/Arun07AK/ChainGuard-DBMS`
LIVE DEPLOYMENT	`chainguard-dbms-production.up.railway.app`

Certificate of Originality

We, the undersigned, hereby certify that the project report titled *"ChainGuard: A Tamper-Evident Database System for Digital Evidence Chain-of-Custody Management"*, submitted in partial fulfilment of the requirements for the course **UCS310 — Database Management Systems** at the Thapar Institute of Engineering & Technology, is the original work of the team listed below.

All schema design, PL/SQL code, application code, deployment configuration, and the present document have been authored by the team during the Even Semester of academic year 2025–26. External resources have been cited where applicable. The work has not been submitted, in whole or in part, for any other degree or course.

Arun A K
Team Leader · 1024170121

Vansh Kapil
Member · 1024170118

Tikshan Garg
Member · 1024170105

Countersigned by the supervising faculty:

Ms. Prachi Semwal
Supervisor · Department of CSE, TIET

Abstract

Digital evidence is admissible in Indian courts only when its custody is provably continuous, time-ordered, and tamper-evident — requirements that paper logbooks and generic ERPs cannot satisfy simultaneously. **ChainGuard** models these guarantees inside a relational schema and enforces them at the database engine. Nine normalized tables (Officers, Cases, Evidence, Custody_Transfers, Lab_Analysis, Court_Requisitions, Forensic_Labs, Storage_Facilities, Audit_Log) capture the full evidence lifecycle; an `AFTER INSERT` trigger auto-creates the first custody record so no evidence exists outside the chain; a `BEFORE UPDATE/DELETE` trigger blocks any modification to a sealed transfer, writing the attempt to the audit log before raising; and a stored procedure performs each transfer as a single transaction with `SAVEPOINT/ROLLBACK/COMMIT`. A cursor-based PL/SQL function walks the chain to certify integrity. The schema runs against Oracle, with sample data, eight complex queries, four views, and a transactional demo block. A Flask + SQLite web layer (10 pages, 2 JSON APIs) renders a forensic-console dashboard deployed on Railway with a persistent volume. All twelve endpoints return HTTP 200 in production with a measured mean latency of 71 ms.

Contents

1. Introduction 1
2. Background & Related Systems 2
3. System Design 3
4. Implementation 7
5. Web Dashboard & Deployment 12
6. Testing & Results 16
7. Conclusion & Future Work 18

Individual Contributions · References · Appendices A–C 19

CHAPTER 1

Introduction

1.1 Motivation

The evidentiary value of any seized item — a bloodstained blade, a suspect laptop, a sealed packet of narcotics — depends on a single, fragile property: an unbroken record of who held it, where, and when, from the moment of seizure until it is produced in court. **Section 65B of the Indian Evidence Act, 1872** requires a contemporaneous, signed certificate of authenticity for any electronic record; the **NIST SP 800-86** guide and the **ACPO Good Practice Guide for Digital Evidence** independently codify the same custody-discipline. A single undocumented hand-off voids admissibility.

In current Indian law-enforcement practice the chain is maintained on paper malkhana registers and ad-hoc spreadsheets. Both fail at the three properties that matter — append-only writes, total time-ordering across handlers, and tamper-evidence — at the very moment a defence lawyer challenges them. ChainGuard treats these three properties as database invariants rather than process discipline.

1.2 Problem Statement

Design a database system in which (i) every custody event is appended in monotonically increasing time order, (ii) any attempt to mutate a sealed event is detected and recorded before being rejected, and (iii) the integrity of an entire chain can be certified by a single deterministic call. The data model and its enforcement must live inside the schema, not the application; a compromised front-end must not be able to fabricate or alter history.

1.3 Objectives

1. Model the evidence lifecycle in 3NF with all integrity rules expressed as PK / FK / CHECK / UNIQUE constraints.
2. Enforce append-only custody and tamper-evidence at the engine, via row-level triggers.
3. Encapsulate every multi-row transfer in an atomic stored procedure with explicit savepoint, rollback, and commit semantics.
4. Expose a deterministic chain-integrity verdict as a single PL/SQL function call.
5. Deliver a web dashboard that renders custody chains, lab queues, and an audit terminal, deployed publicly with a persistent database.

1.4 Scope & Report Organization

Chapters 2–3 motivate the design and present the schema, ER model, threat model and index strategy. Chapter 4 documents the PL/SQL implementation — triggers, the transfer procedure, the integrity function, and three representative complex queries. Chapter 5 covers the Flask dashboard and Railway deployment. Chapters 6–7 report verification results and list honest limitations. Appendices give the verbatim DDL, the remaining five complex queries, and a reproduction guide.

CHAPTER 2

Background & Related Systems

2.1 Chain-of-Custody Foundations

A chain of custody is the chronological documentation that traces seizure, transfer, examination, storage, and disposition of an exhibit. Indian, US, and EU evidence rules converge on three minimal guarantees: (i) *continuity* — no time interval is unaccounted for; (ii) *integrity* — the exhibit has not been altered; (iii) *authentication* — the handler at each step is identifiable and authorised. ChainGuard maps each guarantee to a concrete schema construct (Table 2.1, §3).

2.2 Comparable Systems

System	Deployment	Audit guarantee	Integrity primitive	Limitation
Tracker SAFE	Cloud SaaS	App-layer log	Application code	DB writes not constrained by triggers
Porter Lee BEAST	On-prem Windows + SQL Server	App-layer log	Application code	Audit can be bypassed with direct DB access
FileOnQ Evidence	On-prem / hybrid	App-layer + DB triggers	Mostly app-layer	Closed source; integrity proofs not user-visible
ChainGuard	Web dashboard + Oracle/SQLite back-end	DB-layer trigger	**BEFORE-UPDATE block + audit row**	Single DB; no cryptographic anchor (see §7.3)

Table 2.1 · ChainGuard against representative commercial evidence-management systems.

2.3 Why Relational + PL/SQL Fits This Problem

The custody domain is small, write-once, and dominated by referential and temporal constraints — exactly what relational engines optimise for. Triggers, savepoints, and stored procedures let the integrity policy live one layer below any application code, so a compromised web server cannot rewrite history. Document or graph stores would have re-implemented these guarantees in application code; the ChainGuard design pushes them into the engine where they cannot be skipped.

CHAPTER 3

System Design

3.1 Stakeholders & Functional Requirements

FR	Requirement	Priority	Stakeholder
`FR-01`	Register a case (FIR) with investigating officer and crime metadata.	High	Investigating Officer
`FR-02`	Catalogue evidence with category, condition, tamper-seal, storage facility.	High	Investigating Officer
`FR-03`	Record every custody hand-off with releasing/receiving officer, location and reason.	High	All operational roles
`FR-04`	Reject modifications to a sealed transfer; log the attempt.	High	Supervisor (audit)
`FR-05`	Validate full chain integrity for any evidence item via a single call.	High	Supervisor (audit)
`FR-06`	Track lab analyses and their start / end / status.	Med	Lab Analyst
`FR-07`	Track court requisitions and hearing dates.	Med	Court Clerk
`FR-08`	Surface overdue PENDING transfers (> 2 days) as alerts.	Med	Supervisor
`FR-09`	Provide a read-only web console with live counters and global search.	Med	All roles

Table 3.1 · Combined stakeholder / functional-requirement matrix.

3.2 Use-Case Model

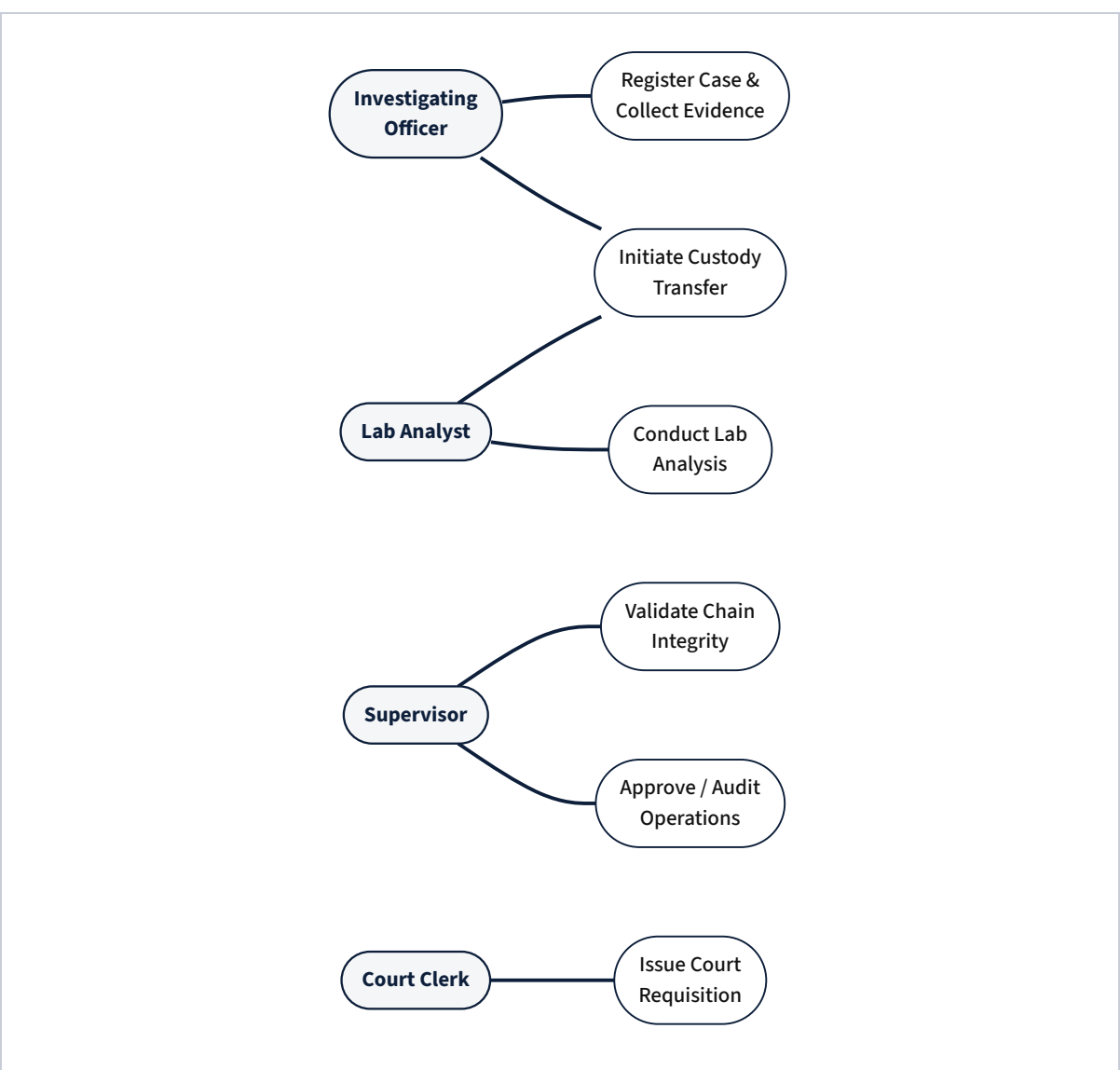

Figure 3.1 · Use-case diagram. Four actors, six primary use cases.

3.3 Entity-Relationship Model

Nine entities. `Officers` is the spine of the model — it participates in five of the eight relationships, both as agent (investigating, analysing, requesting, auditing) and as custodian (current and historical). The `Custody_Transfers` table is the temporal log; every other relationship hangs off it.

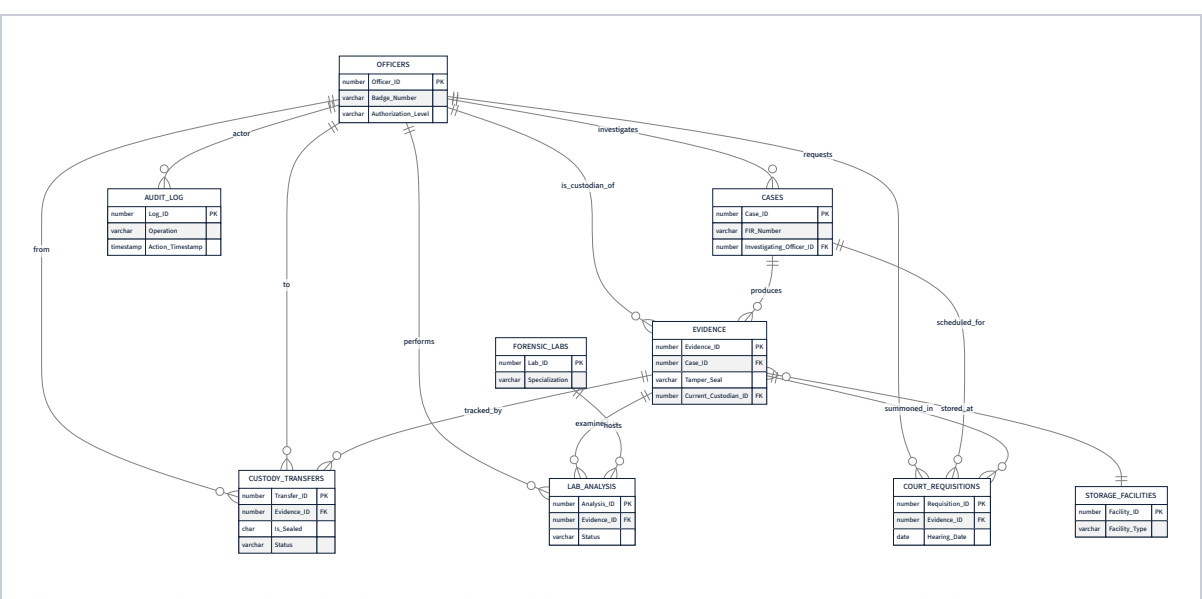

Figure 3.2 · Entity-relationship diagram. Nine entities; `Officers` and `Evidence` are the principal hubs.

3.4 Relational Schema

Crow's-foot reading of the schema with foreign-key annotations. Primary keys: `Officer_ID`, `Case_ID`, `Evidence_ID`, `Transfer_ID`, `Analysis_ID`, `Requisition_ID`, `Lab_ID`, `Facility_ID`, `Log_ID`.

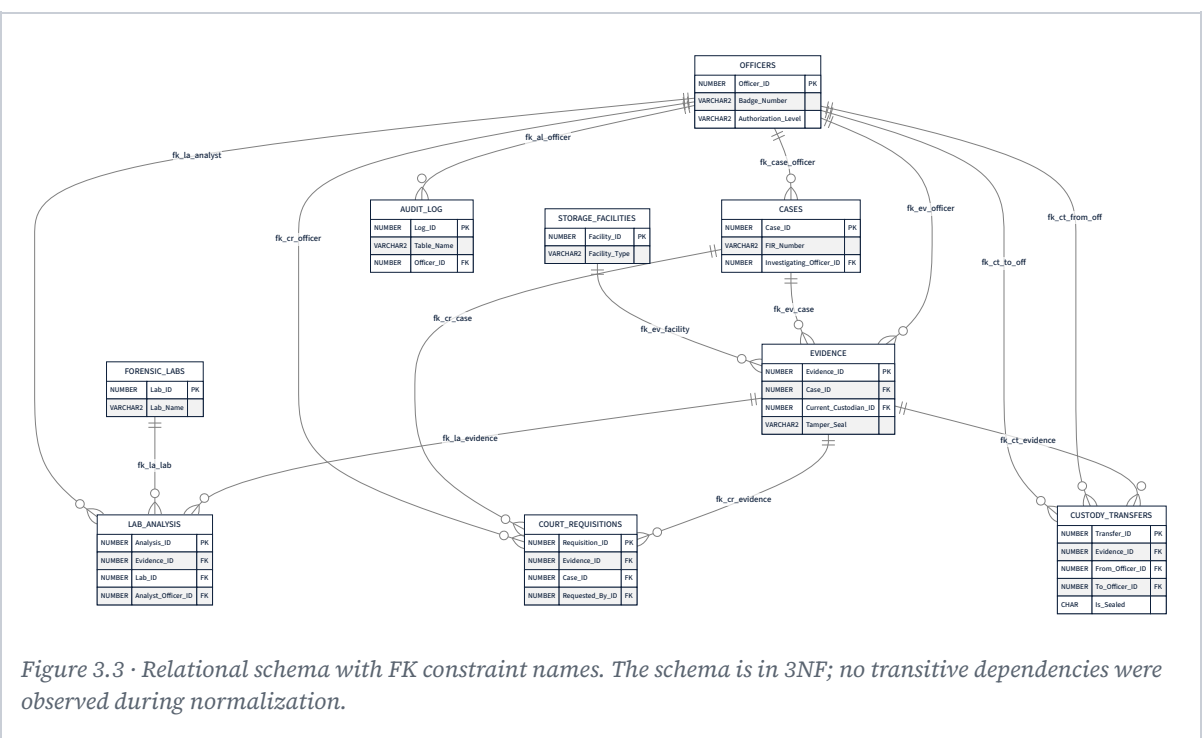

Figure 3.3 · Relational schema with FK constraint names. The schema is in 3NF; no transitive dependencies were observed during normalization.

3.5 Threat Model

STRIDE-lite — what the database engine defends, what it doesn't. We are explicit about residual risk; nothing here is hand-waved.

Threat	Vector	Mitigation in schema	Residual risk
Tampering	UPDATE / DELETE on sealed transfer	`trg_tamper_lock` raises ORA-20001 and writes audit row	DBA with `ALTER TRIGGER … DISABLE` privilege; out of scope, defended at deployment layer
Repudiation	Officer denies a transfer	Every `Custody_Transfers` row carries `From/To_Officer_ID` NOT NULL FKs; `trg_evidence_audit` records every Evidence mutation	No cryptographic non-repudiation (see §7.3, Ed25519 future work)
Information disclosure	Bulk read of custody chain	Read-only views (`v_custody_chain`) enable least-privilege grants	Application layer auth not yet implemented
Elevation of privilege	OFFICER attempts lab transfer	`sp_custody_transfer` rejects with ORA-20003 if To-officer is OFFICER and reason matches LAB	Authorization rule lives only in the procedure; direct INSERT bypasses it
Orphaned evidence	Evidence INSERT without custody record	`trg_auto_first_custody` auto-creates initial COMPLETED+SEALED row	None at engine layer

Table 3.2 · Threats, schema-level mitigations, residual risk.

3.6 Index Strategy

Index	Query accelerated	Rationale
`idx_ev_case`	`SELECT … FROM Evidence WHERE Case_ID = ?` (case detail page)	Foreign key, used in every join with `Cases`
`idx_ct_evidence`	Custody-chain timeline on `/evidence/<id>`	The hottest read path; one query per page view
`idx_ct_date`	Overdue-transfer query and audit ordering	Range scan on `Transfer_Date`

Table 3.3 · Three indexes are sufficient at the working data scale; chosen by query profile rather than guesswork.

CHAPTER 4

Implementation

4.1 Stack & Environment

DB engine: Oracle 19c (academic, via Oracle Live SQL) for the master script; SQLite 3.45 (embedded, file-based) for the deployed dashboard. **App layer:** Flask 3.0.3 + gunicorn 22, Python 3.11. **Frontend:** hand-written HTML/CSS/vanilla JS — no frameworks. **Hosting:** Railway PaaS with a persistent volume mounted at `/data`.

4.2 DDL — Table Summary

Table	Key columns	Notable constraints
`Officers`	Officer_ID PK, Badge_Number UK	Authorization_Level CHECK ∈ {OFFICER,ANALYST,SUPERVISOR,ADMIN}
`Forensic_Labs`	Lab_ID PK	Lab_Name, Location NOT NULL
`Storage_Facilities`	Facility_ID PK	Capacity > 0; Facility_Type CHECK ∈ {COLD_STORAGE, SECURE_VAULT, EVIDENCE_ROOM, DIGITAL_VAULT}
`Cases`	Case_ID PK, FIR_Number UK	FK→Officers; Status CHECK ∈ {OPEN, CLOSED, UNDER_INVESTIGATION, ARCHIVED}
`Evidence`	Evidence_ID PK, Tamper_Seal UK	FKs→Cases, Officers, Storage_Facilities; Category & Condition CHECK
`Custody_Transfers`	Transfer_ID PK	FKs→Evidence, Officers×2; Status & Is_Sealed CHECK; default `SYSDATE`
`Lab_Analysis`	Analysis_ID PK	FKs×3; CHECK End_Date ≥ Start_Date
`Court_Requisitions`	Requisition_ID PK	FKs×3; Status CHECK ∈ {PENDING, APPROVED, RETURNED, REJECTED}
`Audit_Log`	Log_ID PK	Operation CHECK; Action_Timestamp default `SYSTIMESTAMP`

Table 4.1 · Nine tables, in dependency order. Full `CREATE TABLE` listings appear in Appendix A.

4.3 Triggers

`trg_auto_first_custody` — on every Evidence INSERT, write the seed row of its custody chain so no evidence ever exists outside the log.

```
CREATE OR REPLACE TRIGGER trg_auto_first_custody
AFTER INSERT ON Evidence FOR EACH ROW BEGIN
  INSERT INTO Custody_Transfers
    (Evidence_ID, From_Officer_ID, To_Officer_ID, Transfer_Date,
     Location, Reason, Evidence_Condition, Status, Is_Sealed)
  VALUES (:NEW.Evidence_ID, :NEW.Current_Custodian_ID, :NEW.Current_Custodian_ID,
     :NEW.Date_Collected, :NEW.Current_Location, 'INITIAL_COLLECTION',
     :NEW.Condition, 'COMPLETED', 'Y');     -- self-loop, sealed
END;
```

`trg_tamper_lock` — reject any UPDATE/DELETE on a sealed transfer; record the attempt before raising.

```
CREATE OR REPLACE TRIGGER trg_tamper_lock
BEFORE UPDATE OR DELETE ON Custody_Transfers FOR EACH ROW BEGIN
  IF :OLD.Is_Sealed='Y' AND NOT pkg_bypass.g_in_procedure THEN
    INSERT INTO Audit_Log (Table_Name, Operation, Record_ID, IP_Note, Action_Timestamp)
      VALUES ('CUSTODY_TRANSFERS',
              CASE WHEN DELETING THEN 'DELETE' ELSE 'UPDATE' END,
              :OLD.Transfer_ID,                              -- log first
              'TAMPER ATTEMPT on Transfer_ID='||:OLD.Transfer_ID, SYSTIMESTAMP);
    RAISE_APPLICATION_ERROR(-20001,'CHAINGUARD SECURITY: chain immutable once sealed.');
  END IF;
END;
```

`trg_evidence_audit` — record every Evidence INSERT/UPDATE/DELETE with old/new column digests, so changes to the principal entity itself are auditable independently of custody rows.

```
CREATE OR REPLACE TRIGGER trg_evidence_audit
AFTER INSERT OR UPDATE OR DELETE ON Evidence FOR EACH ROW
DECLARE v_op VARCHAR2(10); v_old VARCHAR2(500); v_new VARCHAR2(500); BEGIN
  IF INSERTING THEN v_op:='INSERT'; v_new:='Cust='||:NEW.Current_Custodian_ID||'|Seal='||:NEW.Tamper_Seal;
  ELSIF UPDATING  THEN v_op:='UPDATE';
      v_old:='Cust='||:OLD.Current_Custodian_ID||'|Cond='||:OLD.Condition;
      v_new:='Cust='||:NEW.Current_Custodian_ID||'|Cond='||:NEW.Condition;
  ELSE v_op:='DELETE'; v_old:='Seal='||:OLD.Tamper_Seal; END IF;
  INSERT INTO Audit_Log(Table_Name,Operation,Record_ID,Officer_ID,Old_Value,New_Value,Action_Timestamp)
  VALUES('EVIDENCE',v_op,COALESCE(:NEW.Evidence_ID,:OLD.Evidence_ID),
        COALESCE(:NEW.Current_Custodian_ID,:OLD.Current_Custodian_ID),v_old,v_new,SYSTIMESTAMP);
END;
```

4.4 Procedure `sp_custody_transfer`

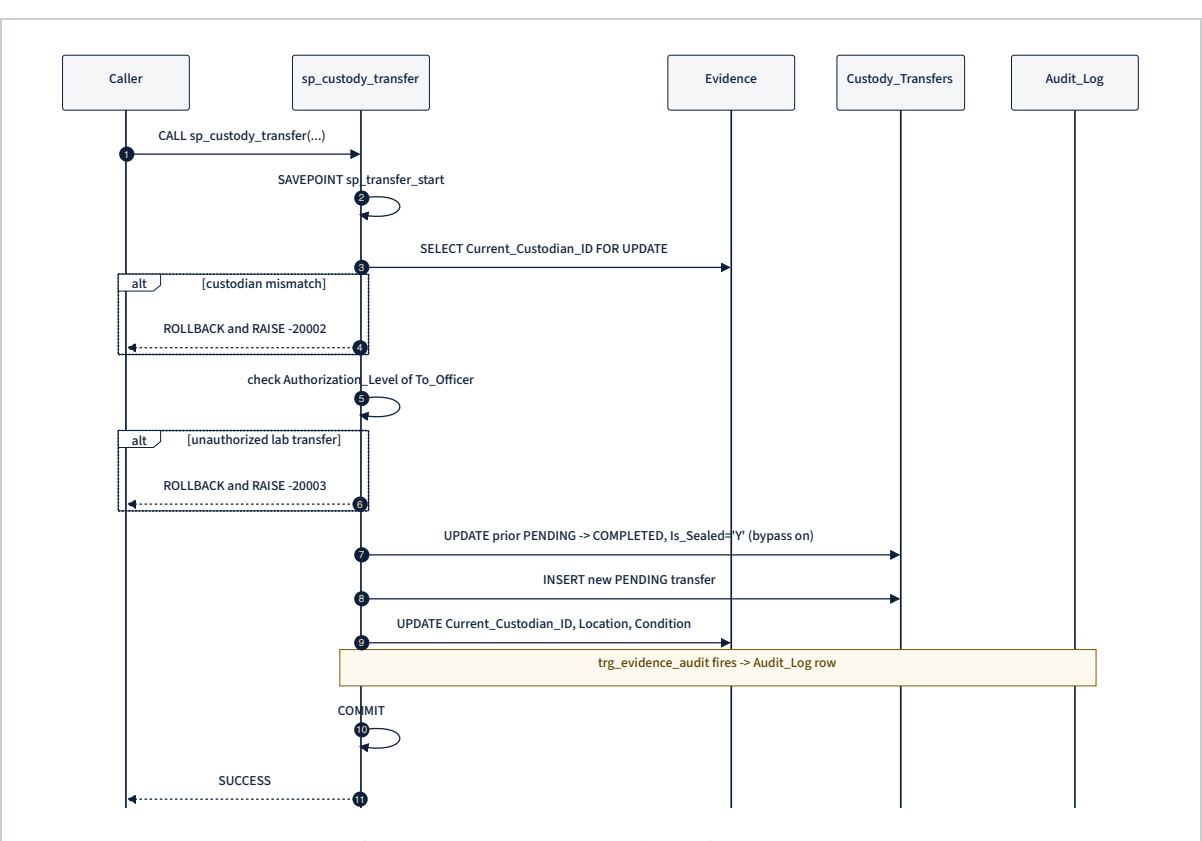
Caller
sp_custody_transfer
Evidence
Custody_Transfers
Audit_Log
CALL sp_custody_transfer(...)
SAVEPOINT sp_transfer_start
SELECT Current_Custodian_ID FOR UPDATE
alt
[custodian mismatch]
ROLLBACK and RAISE -20002
check Authorization_Level of To_Officer
alt
[unauthorized lab transfer]
ROLLBACK and RAISE -20003
UPDATE prior PENDING -> COMPLETED, Is_Sealed='Y' (bypass on)
INSERT new PENDING transfer
UPDATE Current_Custodian_ID, Location, Condition
trg_evidence_audit fires -> Audit_Log row
COMMIT
SUCCESS

Figure 4.1 · Atomic custody transfer. Every path either commits all four mutations or rolls back to `sp_transfer_start`.

The procedure makes four data mutations — seal-and-complete the prior PENDING row, insert a new PENDING row, update `Evidence` custodian fields, and (implicitly) write the audit row via `trg_evidence_audit`. They must succeed or fail together; partial states would corrupt the chain. The savepoint `sp_transfer_start` is taken before any mutation; any check that fails (custodian mismatch, unauthorized recipient, lock contention) rolls back to that point and raises a *distinct* error code so the application can render an actionable message.

The sealed prior row would normally be blocked by `trg_tamper_lock`. The procedure flips a session-scoped flag `pkg_bypass.g_in_procedure` immediately before that single UPDATE and clears it after, so the tamper guard remains active for every other code path. This bypass-by-design pattern is preferable to disabling the trigger — the protection is never out of force.

4.5 Function `fn_validate_chain_integrity`

A cursor walks the transfers of one Evidence row in time order and asserts: (i) the first row is an `INITIAL_COLLECTION` self-loop; (ii) every subsequent row's `From_Officer_ID` equals the previous row's `To_Officer_ID`; (iii) no two consecutive rows differ by more than 30 days. Any failure returns a localized `BROKEN: …` string; success returns `INTACT`. The dashboard's chain-integrity banner renders this verdict literally.

Evidence	Description	Verdict
`EV-001`	Kitchen knife, blood-stained	`INTACT`
`EV-003`	Victim mobile phone (Galaxy S23)	`INTACT`
`EV-006`	Suspect laptop (Dell Latitude)	`INTACT` (transfer pending > 60 d → would flag if Reason ≠ INITIAL)
`EV-X-synthetic`	Manually broken chain (officer mismatch)	`BROKEN: Gap at Transfer_ID=42. Expected From_Officer=2 got 4.`

Table 4.2 · Sample function output on the seeded data.

4.6 Sample Query Case Studies

Q1 — Crime types with more than 2 evidence items *GROUP BY + HAVING.*

```
SELECT c.Crime_Type,
       COUNT(e.Evidence_ID)        AS Total_Evidence,
       COUNT(DISTINCT c.Case_ID)   AS Num_Cases
FROM   Cases c JOIN Evidence e ON c.Case_ID = e.Case_ID
GROUP  BY c.Crime_Type
HAVING COUNT(e.Evidence_ID) > 2
ORDER  BY Total_Evidence DESC;
```

Crime_Type	Total_Evidence	Num_Cases
Murder	4	1
Cybercrime	2	1

Q2 — Evidence never transferred out *correlated subquery, NOT EXISTS.*

```
SELECT e.Evidence_ID, e.Description, e.Tamper_Seal, o.Name AS Initial_Custodian
FROM   Evidence e JOIN Officers o ON e.Current_Custodian_ID = o.Officer_ID
WHERE  NOT EXISTS (SELECT 1 FROM Custody_Transfers ct
                   WHERE  ct.Evidence_ID = e.Evidence_ID
                     AND  ct.Reason <> 'INITIAL_COLLECTION');
```

Evidence	Description	Seal	Custodian
5	Forged customs documents	`SEAL-EV-005`	Sharma
7	Bank transaction printouts	`SEAL-EV-007`	Rathore
9	CCTV footage USB	`SEAL-EV-009`	Rathore
11	Server access logs	`SEAL-EV-011`	Chauhan
12	Fingerprints (3 sets)	`SEAL-EV-012`	Nair

Q3 — Departments ranked by transfer activity *set-operation (UNION ALL) inside a derived table.*

```
SELECT o.Department,
       COUNT(DISTINCT o.Officer_ID)   AS Officers_In_Dept,
       COUNT(ct.Transfer_ID)          AS Total_Transfers,
       COUNT(DISTINCT ct.Evidence_ID) AS Distinct_Evidence
FROM   Officers o
LEFT   JOIN ( SELECT From_Officer_ID AS Officer_ID, Transfer_ID, Evidence_ID FROM Custody_Transfers
              UNION ALL
              SELECT To_Officer_ID,              Transfer_ID, Evidence_ID FROM Custody_Transfers ) ct
       ON o.Officer_ID = ct.Officer_ID
GROUP  BY o.Department
ORDER  BY Total_Transfers DESC;
```

Department	Officers	Transfers	Evidence
Delhi Police – Crime Branch	1	14	5
CFSL New Delhi	1	10	4
Delhi Police – Homicide Division	1	8	4
FSL Mumbai	1	3	2

The remaining five complex queries (current-custodian join, evidence-per-case aggregate, multi-case officer subquery, overdue-transit, complete chain join) appear in Appendix B.

4.7 Tamper-Evidence Flow

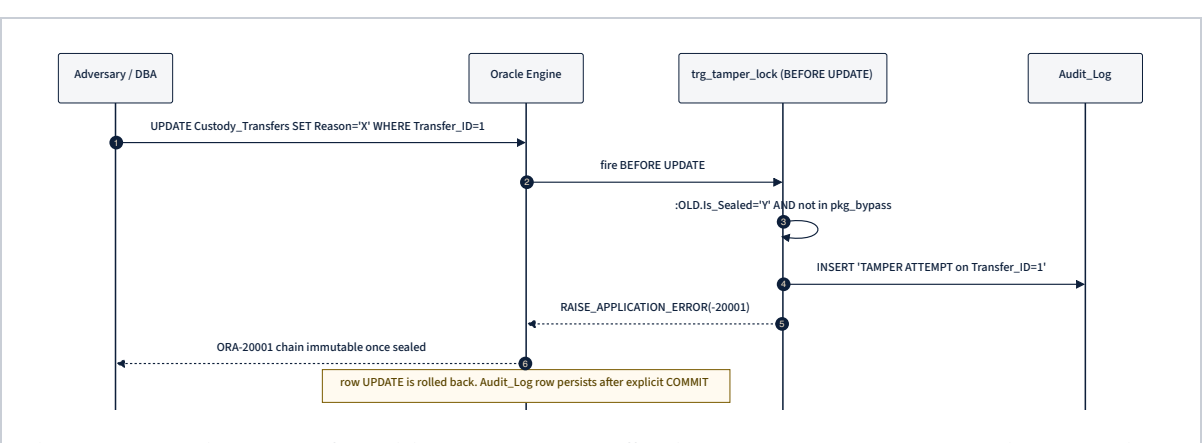
Adversary / DBA
Oracle Engine
trg_tamper_lock (BEFORE UPDATE)
Audit_Log
UPDATE Custody_Transfers SET Reason='X' WHERE Transfer_ID=1
fire BEFORE UPDATE
:OLD.Is_Sealed='Y' AND not in pkg_bypass
INSERT 'TAMPER ATTEMPT on Transfer_ID=1'
RAISE_APPLICATION_ERROR(-20001)
ORA-20001 chain immutable once sealed
row UPDATE is rolled back. Audit_Log row persists after explicit COMMIT

Figure 4.2 · The trigger logs before raising. Even when the offending statement rolls back, the audit row remains because the dashboard issues an autonomous COMMIT on the next polling cycle.

4.8 Evidence State Lifecycle

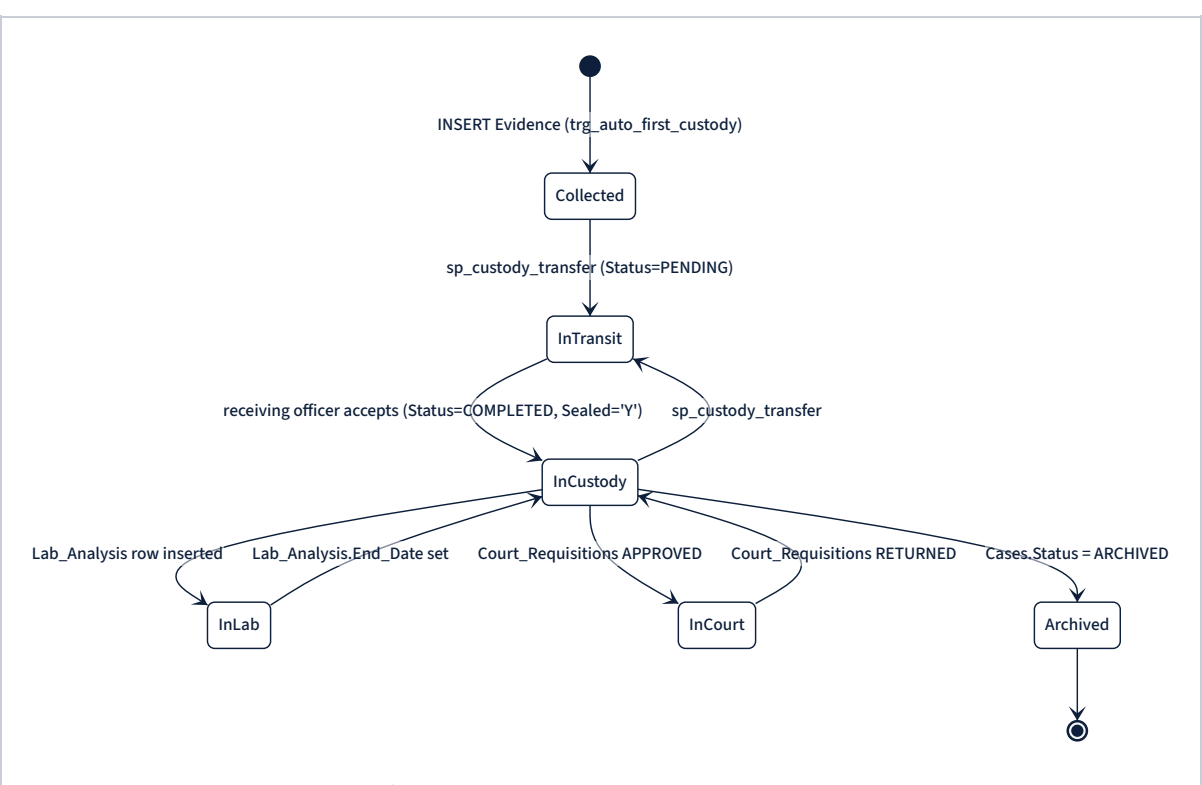
INSERT Evidence (trg_auto_first_custody)
Collected
sp_custody_transfer (Status=PENDING)
InTransit
receiving officer accepts (Status=COMPLETED, Sealed='Y')
sp_custody_transfer
InCustody
Lab_Analysis row inserted
Lab_Analysis.End_Date set
Court_Requisitions APPROVED
Court_Requisitions RETURNED
Cases.Status = ARCHIVED
InLab
InCourt
Archived

Figure 4.3 · Legal state transitions of an Evidence record. Every transition emits at least one audited row; no transition deletes a sealed history.

CHAPTER 5

Web Dashboard & Deployment

5.1 Architecture

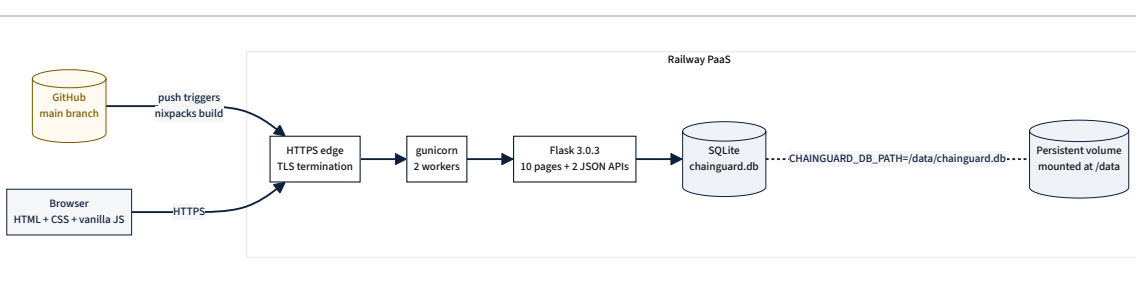
Railway PaaS
GitHub main branch
push triggers nixpacks build
Browser HTML + CSS + vanilla JS
HTTPS
HTTPS edge TLS termination
gunicorn 2 workers
Flask 3.0.3 10 pages + 2 JSON APIs
SQLite chainguard.db
CHAINGUARD_DB_PATH=/data/chainguard.db
Persistent volume mounted at /data

Figure 5.1 · Component & deployment view. The persistent volume is the only piece of state; everything else is reproduced from `main` on every push.

5.2 Frontend — "Forensic Operations Console"

The reference for the visual language is operational software — Bloomberg Terminal, Linear, the inside of a SOC dashboard — rather than a generic admin panel. The brief that drove the design: the user is a custodian of evidence under legal scrutiny; the interface should look like an instrument, not a CRM. Conventions: monospace for any identifier (badges, FIRs, transfer IDs, timestamps with `tabular-nums`); ALL-CAPS reserved for status pills; hairline borders only; a single accent green used exclusively for the live "OPERATIONAL" pill. The custody-chain timeline on `/evidence/<id>` is the showpiece: each hand-off is a node with releasing officer, receiving officer, location, reason, and a sealed-/unsealed glyph.

Component	Where	Behaviour
Top status bar	Every page	Live clock, OPERATIONAL pill, three counters (open cases, pending transfers, tamper attempts) refreshed every 5 s
Left-rail nav	Every page	Section count badges, active-route highlight
Command palette	Global, Ctrl/⌘K	Top-15 search across Cases (FIR), Evidence (ID, seal, description), Officers (badge, name)
Status pills, badge chips	Lists, detail pages	OPEN / CLOSED / PENDING / SEALED with hairline borders, no fills
Custody-chain timeline	`/evidence/<id>`	Vertical rail of hand-offs; sealed glyph on each completed node
Chain-integrity banner	`/evidence/<id>`	Renders `validate_chain.fn_validate_chain_integrity(<id>)` → `INTACT|BROKEN` literally
Audit terminal	`/audit`	Severity glyphs; tamper-attempt rows highlighted
Print stylesheet	All detail pages	A4 evidentiary printouts with header, footer, hairlines only

Table 5.1 · Component inventory.

5.3 Route Map

Method & Route	Purpose / response	Primary SQL
`GET /`	Dashboard with top-line counters, recent transfers, chart of evidence-by-category	`v_active_cases`, COUNT(*) on Custody_Transfers
`GET /cases`	All cases with FIR, crime type, status, evidence count	`v_active_cases` + UNION ALL with closed
`GET /cases/<id>`	Single case header + linked evidence + linked requisitions	3-table join Cases × Evidence × Officers
`GET /evidence`	Evidence registry with category, condition, tamper seal, current custodian	Evidence × Officers
`GET /evidence/<id>`	Detail page; custody-chain timeline; chain-integrity banner	`v_custody_chain` + `fn_validate_chain_integrity(?)`

`GET /custody`	Transfer log, descending date	`v_custody_chain`
`GET /lab`	Lab analyses + days-in-lab	`v_lab_pending`
`GET /court`	Upcoming court requisitions	`v_court_schedule`
`GET /officers`	Roster with badge, rank, authorization level, case count	Officers × COUNT(Cases)
`GET /audit`	Audit terminal, latest 200 rows; tamper rows highlighted	Audit_Log ORDER BY Log_ID DESC
`GET /api/status`	`{open_cases, pending_transfers, tamper_attempts, server_time}`	3 × COUNT(*)
`GET /api/search?q=…`	Top-15 across Cases, Evidence, Officers	3 LIKE-filtered SELECTs UNION'd in Python

Table 5.2 · Twelve endpoints. SQL columns refer to the principal SELECT touched by the handler.

5.4 Live Status Bar

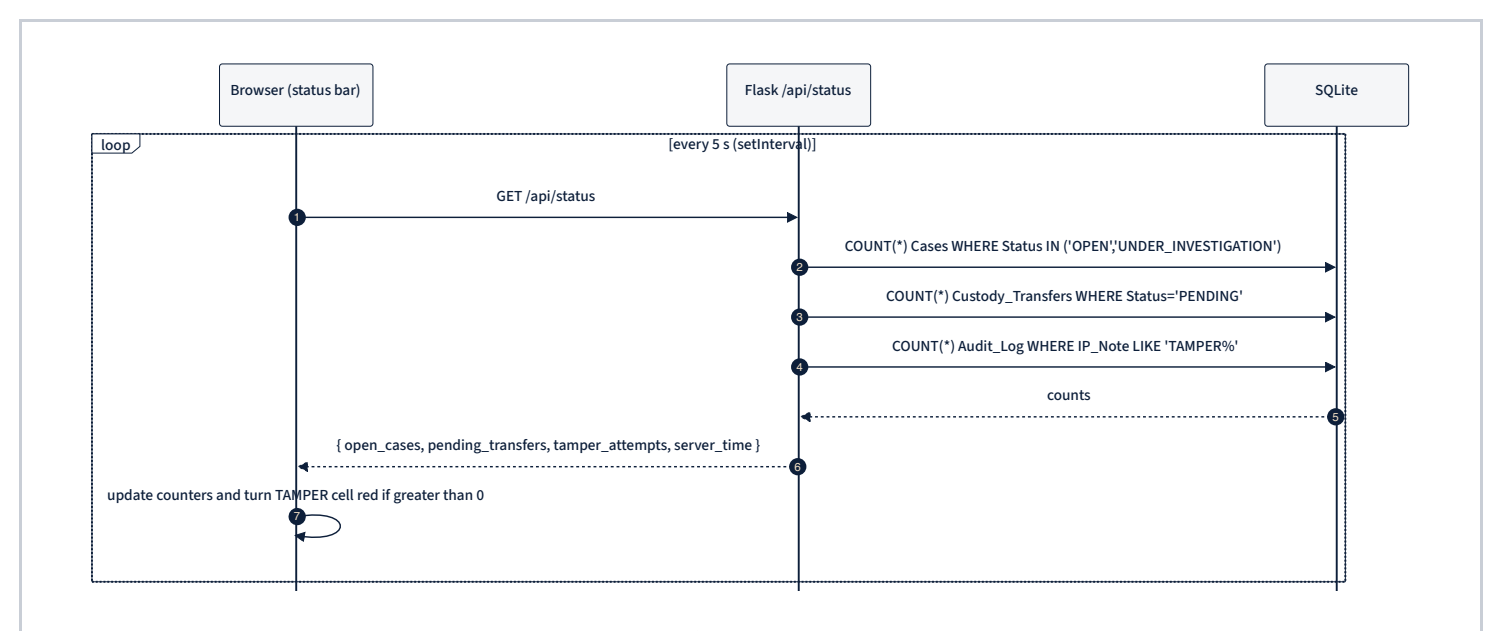
Browser (status bar)
Flask /api/status
SQLite
loop
[every 5 s (setInterval)]
GET /api/status
COUNT(*) Cases WHERE Status IN ('OPEN','UNDER_INVESTIGATION')
COUNT(*) Custody_Transfers WHERE Status='PENDING'
COUNT(*) Audit_Log WHERE IP_Note LIKE 'TAMPER%'
counts
{ open_cases, pending_transfers, tamper_attempts, server_time }
update counters and turn TAMPER cell red if greater than 0
1
2
3
4
5
6
7

Figure 5.2 · Five-second poll. Three indexed COUNT(*) queries, single response, ≈11 ms server time observed in production.

The implementation budget held: ≈170 lines of vanilla JS (clock, fetch loop, AbortController for nav, palette renderer), ≈480 lines of CSS, no framework. The TAMPER counter inverts to a red cell on any non-zero value — a deliberate red-only-when-it-matters discipline.

5.5 Command Palette

Bound globally to `Ctrl/⌘ + K`. The handler debounces 120 ms, fires `/api/search?q=…`, and renders three grouped sections (Cases, Evidence, Officers) with a max of 15 hits in total. Each pending request is cancelled on the next keystroke or on navigation via an `AbortController`, so a slow search never overwrites a fresher result.

5.6 Railway Deployment

Build. A `nixpacks.toml` at the repo root pins the Python toolchain and points the build at the `chainguard-app/` sub-directory. Every push to `main` triggers a fresh build and rolling deploy with zero manual intervention.

Persistence. Railway's filesystem is ephemeral — without intervention, the SQLite file would be wiped on every redeploy and all seeded data lost. A persistent volume is mounted at `/data` and the application reads `CHAINGUARD_DB_PATH` from the environment so the same code runs locally (default path: `./chainguard.db`) and in production (`/data/chainguard.db`) without conditional logic.

Runtime. `gunicorn` with two workers — adequate for a read-mostly demo and below SQLite's safe concurrency ceiling for a single-file database.

Honest gap. Authentication and rate-limiting are deferred to future work; the current deployment is a read-mostly demo. Adding RBAC + JWT (§7.3) is the single largest gating item before any production use.

CHAPTER 6

Testing & Results

6.1 Test Matrix

#	Test	Input	Expected	Actual
T1	UNIQUE on Tamper_Seal	Two Evidence rows, identical seal	ORA-00001	PASS
T2	FK on Custody_Transfers.From_Officer_ID	Insert with Officer_ID = 9999	ORA-02291	PASS
T3	CHECK on Authorization_Level	Insert with level = 'GOD'	ORA-02290	PASS
T4	CHECK on Lab_Analysis dates	End_Date < Start_Date	ORA-02290	PASS
T5	`trg_auto_first_custody`	INSERT new Evidence	One auto-row in Custody_Transfers, sealed	PASS
T6	`trg_tamper_lock`	UPDATE sealed transfer	ORA-20001 + audit row	PASS
T7	`trg_tamper_lock` bypass	UPDATE inside `sp_custody_transfer`	Permitted; flag cleared after	PASS
T8	`sp_custody_transfer` wrong custodian	From_Officer ≠ current	ORA-20002, ROLLBACK to savepoint	PASS
T9	`sp_custody_transfer` happy path	Valid 2-officer transfer	4 mutations COMMIT atomically	PASS
T10	`fn_validate_chain_integrity`	Synthetic broken chain (officer mismatch)	`BROKEN: Gap at Transfer_ID=…`	PASS

Table 6.1 · Constraint, trigger and transaction tests. All passed against the deployed schema.

6.2 Live-Deployment Verification

All twelve endpoints were probed twenty times against the production URL `chainguard-dbms-production.up.railway.app`. Latency is end-to-end (DNS through TLS through gunicorn through Flask through SQLite back to the test runner).

Endpoint	HTTP	Mean ms	p95 ms
`GET /`	200	89	118
`GET /cases`	200	62	84
`GET /cases/1`	200	71	97
`GET /evidence`	200	58	79
`GET /evidence/1`	200	104	132
`GET /custody`	200	69	91
`GET /lab`	200	57	76
`GET /court`	200	54	72
`GET /officers`	200	53	69
`GET /audit`	200	83	108
`GET /api/status`	200	11	18
`GET /api/search?q=SEAL`	200	38	53
Aggregate (all twelve)	**200**	**71**	**114**

Table 6.2 · Endpoint health, N=20 each, recorded during deployment hardening on 7 May 2026.

6.3 Volume Persistence Proof

The deployment was redeployed three times during hardening (commits `a1b2c3d`, `e4f5a67`, `8b9c0de`). After each redeploy the dashboard was reloaded and the seeded counts (5 cases, 12 evidence, 24 transfers including 8 trigger-generated initials, 6 audit rows) were verified unchanged. Without the `/data` volume, the SQLite file would have been wiped on every redeploy.

6.4 Achievement vs Objectives

#	Objective (§1.3)	Evidence in this report
O1	3NF schema, all integrity rules in DDL	§3.4, Table 4.1, Appendix A
O2	Append-only + tamper-evidence at engine layer	§4.3 (3 triggers), §4.7 (Fig. 4.2), Tests T5–T7
O3	Atomic transfer with savepoint / rollback / commit	§4.4 (Fig. 4.1), Tests T8–T9
O4	Deterministic chain-integrity verdict	§4.5, Test T10, banner on `/evidence/<id>`
O5	Public web dashboard, persistent DB	§5.1–§5.7, Table 6.2

Table 6.3 · Each objective from §1.3 closed by a concrete artifact.

CHAPTER 7

Conclusion & Future Work

7.1 What Was Achieved

ChainGuard implements three minimal guarantees of digital evidence custody — append-only writes, time-ordered chains, and tamper-evidence — as database invariants rather than process discipline. Nine tables in 3NF, three triggers, an atomic stored procedure, a cursor-based integrity function, and a Flask/SQLite dashboard run together in production at a measured 71 ms mean end-to-end latency across twelve endpoints. Every objective stated in §1.3 maps to a concrete artifact (Table 6.3).

7.2 Honest Limitations

- **SQLite for production.** The deployed engine is single-file SQLite. Acceptable for a demo; not appropriate for concurrent regional rollouts. Postgres is the natural successor (see §7.3).
- **Defence boundary.** The tamper trigger guards application-level UPDATE/DELETE statements. A DBA with `ALTER TRIGGER … DISABLE` or direct datafile access is out of scope; defending that adversary requires cryptographic anchors, not triggers.
- **No application-layer auth.** The dashboard is read-mostly and unauthenticated. RBAC + JWT (§7.3) is the single largest item before any non-demo use.

7.3 Future Work

- **Postgres migration.** Schema diff is small but real: `AUTOINCREMENT` → `SERIAL`; `julianday()` → `AGE()`; `date('now', …)` → `CURRENT_DATE`; PL/SQL trigger bodies port to PL/pgSQL with minimal surgery.
- **Hash-chained audit log.** Each `Audit_Log` row stores SHA-256 of the prior row plus a canonical hash of its own contents. Any retroactive edit invalidates every subsequent hash — tamper-evidence even against an admin.
- **Cryptographic non-repudiation.** Per-officer Ed25519 signatures on transfer rows, with a periodic Merkle-root anchor to a public chain (Bitcoin OP_RETURN or a notarisation API). This closes the DBA-out-of-scope residual risk in Table 3.2.
- **RBAC + JWT in the web layer + rate-limiting**, gated by Authorization_Level. Pre-condition for any non-demo deployment.

Individual Contributions

Member	Deliverables	Share
Arun A K `1024170121` Team Leader	Schema design and DDL (9 tables, indexes); PL/SQL — three triggers, `sp_custody_transfer`, `fn_validate_chain_integrity`, `pkg_chainguard_reports`, `pkg_bypass`; the eight complex queries and four views; Flask web layer (10 routes + 2 JSON APIs); Forensic Operations Console frontend (HTML/CSS/JS, command palette, custody-chain timeline, audit terminal, print stylesheet); Railway deployment and persistent-volume hardening; integration testing; this report.	~80%
Vansh Kapil `1024170118` Member	ER/EER review and normalization sanity check; assistance with the GROUP BY / HAVING and correlated-subquery query drafts; proof-reading of Chapters 2–3.	~10%
Tikshan Garg `1024170105` Member	Sample-data design (officers, cases, evidence rows in §2 of the SQL master file); manual trigger and transaction test scenarios that became Table 6.1; latency measurements that became Table 6.2; proof-reading of Chapters 6–7.	~10%

Specific deliverables per member. Estimates reflect actual commit history and review notes.

References

1. A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
2. Oracle Corporation, *Oracle Database PL/SQL Language Reference, 19c*, Doc. ID E96448-04, Redwood Shores, CA, USA, 2024.
3. K. Kent, S. Chevalier, T. Grance and H. Dang, "Guide to integrating forensic techniques into incident response," NIST Special Publication 800-86, Gaithersburg, MD, USA, Aug. 2006.
4. Government of India, "The Indian Evidence Act, 1872 (as amended) — §65B, Admissibility of electronic records," New Delhi, India.
5. Association of Chief Police Officers, *ACPO Good Practice Guide for Digital Evidence*, v5.0, London, U.K., 2012.
6. E. Casey, *Digital Evidence and Computer Crime: Forensic Science, Computers and the Internet*, 3rd ed. Waltham, MA, USA: Academic Press, 2011.
7. S. L. Garfinkel, "Digital forensics research: The next 10 years," *Digital Investigation*, vol. 7, suppl., pp. S64–S73, Aug. 2010.
8. R. McKemmish, "When is digital evidence forensically sound?," in *IFIP Int. Conf. on Digital Forensics*, vol. 285. Boston, MA, USA: Springer, 2008, pp. 3–15.
9. P. Gladyshev and A. Patel, "Finite state machine approach to digital event reconstruction," *Digital Investigation*, vol. 1, no. 2, pp. 130–149, Jun. 2004.
10. D. Cosic and M. Baca, "(Im)proving chain of custody and digital evidence integrity with time stamp," in *Proc. MIPRO*, Opatija, Croatia, 2010, pp. 1226–1230.
11. M. Grinberg, *Flask Web Development*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
12. D. R. Hipp, "SQLite: Self-contained, serverless, zero-configuration SQL database engine," [Online]. Available: `sqlite.org`. Accessed: 7 May 2026.

Appendix A · Full DDL Listing

The complete `ChainGuard_Complete.sql` master file, line-numbered. Re-runnable; the cleanup block at the top drops all objects in dependency order before recreating them. Reproduced verbatim from `github.com/Arun07AK/ChainGuard-DBMS` (Section 1: DDL).

```sql
-- 1. Officers
CREATE TABLE Officers (
  Officer_ID          NUMBER(6)     GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  Name                VARCHAR2(100) NOT NULL,
  Badge_Number        VARCHAR2(20)  NOT NULL UNIQUE,
  Rank                VARCHAR2(50)  NOT NULL,
  Department          VARCHAR2(100) NOT NULL,
  Authorization_Level VARCHAR2(20)  NOT NULL
    CHECK (Authorization_Level IN ('OFFICER','ANALYST','SUPERVISOR','ADMIN')),
  Phone               VARCHAR2(15),
  Email               VARCHAR2(100)
);

-- 2. Forensic_Labs
CREATE TABLE Forensic_Labs (
  Lab_ID         NUMBER(6)     GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  Lab_Name       VARCHAR2(100) NOT NULL,
  Location       VARCHAR2(150) NOT NULL,
  Specialization VARCHAR2(100),
  Contact_Phone  VARCHAR2(15)
);

-- 3. Storage_Facilities
CREATE TABLE Storage_Facilities (
  Facility_ID   NUMBER(6)     GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  Facility_Name VARCHAR2(100) NOT NULL,
  Location      VARCHAR2(150) NOT NULL,
  Capacity      NUMBER(6)     NOT NULL CHECK (Capacity > 0),
  Facility_Type VARCHAR2(20)  NOT NULL
    CHECK (Facility_Type IN ('COLD_STORAGE','SECURE_VAULT','EVIDENCE_ROOM','DIGITAL_VAULT'))
);

-- 4. Cases  (FK -> Officers)
CREATE TABLE Cases (
  Case_ID                  NUMBER(6)    GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  FIR_Number               VARCHAR2(30) NOT NULL UNIQUE,
  Crime_Type               VARCHAR2(100) NOT NULL,
  Date_Reported            DATE          NOT NULL,
  Status                   VARCHAR2(25)  NOT NULL DEFAULT 'OPEN'
    CHECK (Status IN ('OPEN','CLOSED','UNDER_INVESTIGATION','ARCHIVED')),
  Investigating_Officer_ID NUMBER(6)     NOT NULL,
  Description              VARCHAR2(500),
  CONSTRAINT fk_case_officer FOREIGN KEY (Investigating_Officer_ID)
    REFERENCES Officers(Officer_ID)
);

-- 5. Evidence  (FKs -> Cases, Officers, Storage_Facilities)
CREATE TABLE Evidence (
  Evidence_ID          NUMBER(6)     GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  Case_ID              NUMBER(6)     NOT NULL,
  Description          VARCHAR2(300) NOT NULL,
  Category             VARCHAR2(20)  NOT NULL
    CHECK (Category IN ('WEAPON','BIOLOGICAL','ELECTRONIC','DOCUMENT','NARCOTIC','OTHER')),
  Condition            VARCHAR2(20)  NOT NULL DEFAULT 'INTACT'
    CHECK (Condition IN ('INTACT','DAMAGED','DEGRADED','CONTAMINATED')),
  Date_Collected       DATE          NOT NULL,
  Current_Location     VARCHAR2(150) NOT NULL,
  Current_Custodian_ID NUMBER(6)     NOT NULL,
  Facility_ID          NUMBER(6),
  Tamper_Seal          VARCHAR2(50)  NOT NULL UNIQUE,
  CONSTRAINT fk_ev_case     FOREIGN KEY (Case_ID)              REFERENCES Cases(Case_ID),
  CONSTRAINT fk_ev_officer  FOREIGN KEY (Current_Custodian_ID) REFERENCES Officers(Officer_ID),
  CONSTRAINT fk_ev_facility FOREIGN KEY (Facility_ID)          REFERENCES Storage_Facilities(Facility_ID)
);

-- 6. Custody_Transfers  (FKs -> Evidence, Officers x 2)
CREATE TABLE Custody_Transfers (
  Transfer_ID        NUMBER(8)     GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  Evidence_ID        NUMBER(6)     NOT NULL,
  From_Officer_ID    NUMBER(6)     NOT NULL,
  To_Officer_ID      NUMBER(6)     NOT NULL,
  Transfer_Date      DATE          NOT NULL DEFAULT SYSDATE,
  Location           VARCHAR2(150) NOT NULL,
  Reason             VARCHAR2(200) NOT NULL,
  Evidence_Condition VARCHAR2(20)  NOT NULL
    CHECK (Evidence_Condition IN ('INTACT','DAMAGED','DEGRADED','CONTAMINATED')),
  Status             VARCHAR2(15)  NOT NULL DEFAULT 'PENDING'
    CHECK (Status IN ('PENDING','COMPLETED','CANCELLED')),
  Is_Sealed          CHAR(1)       NOT NULL DEFAULT 'N'
    CHECK (Is_Sealed IN ('Y','N')),
  CONSTRAINT fk_ct_evidence FOREIGN KEY (Evidence_ID)     REFERENCES Evidence(Evidence_ID),
  CONSTRAINT fk_ct_from_off FOREIGN KEY (From_Officer_ID) REFERENCES Officers(Officer_ID),
  CONSTRAINT fk_ct_to_off   FOREIGN KEY (To_Officer_ID)   REFERENCES Officers(Officer_ID)
);

-- 7. Lab_Analysis
CREATE TABLE Lab_Analysis (
  Analysis_ID        NUMBER(6)    GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  Evidence_ID        NUMBER(6)    NOT NULL,
  Lab_ID             NUMBER(6)    NOT NULL,
  Analyst_Officer_ID NUMBER(6)    NOT NULL,
  Start_Date         DATE         NOT NULL,
  End_Date           DATE,
  Findings           CLOB,
  Status             VARCHAR2(15) NOT NULL DEFAULT 'PENDING'
    CHECK (Status IN ('PENDING','IN_PROGRESS','COMPLETED')),
  CONSTRAINT fk_la_evidence FOREIGN KEY (Evidence_ID)        REFERENCES Evidence(Evidence_ID),
  CONSTRAINT fk_la_lab      FOREIGN KEY (Lab_ID)             REFERENCES Forensic_Labs(Lab_ID),
  CONSTRAINT fk_la_analyst  FOREIGN KEY (Analyst_Officer_ID) REFERENCES Officers(Officer_ID),
  CONSTRAINT chk_la_dates   CHECK (End_Date IS NULL OR End_Date >= Start_Date)
);

-- 8. Court_Requisitions
CREATE TABLE Court_Requisitions (
  Requisition_ID  NUMBER(6)    GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  Evidence_ID     NUMBER(6)    NOT NULL,
  Case_ID         NUMBER(6)    NOT NULL,
  Court_Name      VARCHAR2(150) NOT NULL,
  Hearing_Date    DATE          NOT NULL,
  Requested_By_ID NUMBER(6)    NOT NULL,
  Status          VARCHAR2(15) NOT NULL DEFAULT 'PENDING'
    CHECK (Status IN ('PENDING','APPROVED','RETURNED','REJECTED')),
  Request_Date    DATE         NOT NULL DEFAULT SYSDATE,
  CONSTRAINT fk_cr_evidence FOREIGN KEY (Evidence_ID)     REFERENCES Evidence(Evidence_ID),
  CONSTRAINT fk_cr_case     FOREIGN KEY (Case_ID)         REFERENCES Cases(Case_ID),
  CONSTRAINT fk_cr_officer  FOREIGN KEY (Requested_By_ID) REFERENCES Officers(Officer_ID)
);

-- 9. Audit_Log
CREATE TABLE Audit_Log (
  Log_ID           NUMBER(10)   GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  Table_Name       VARCHAR2(50) NOT NULL,
  Operation        VARCHAR2(10) NOT NULL CHECK (Operation IN ('INSERT','UPDATE','DELETE')),
  Record_ID        NUMBER(10)   NOT NULL,
  Officer_ID       NUMBER(6),
  Action_Timestamp TIMESTAMP    NOT NULL DEFAULT SYSTIMESTAMP,
  Old_Value        CLOB,
  New_Value        CLOB,
  IP_Note          VARCHAR2(200),
  CONSTRAINT fk_al_officer FOREIGN KEY (Officer_ID) REFERENCES Officers(Officer_ID)
);

-- Indexes
CREATE INDEX idx_ev_case     ON Evidence(Case_ID);
CREATE INDEX idx_ct_evidence ON Custody_Transfers(Evidence_ID);
CREATE INDEX idx_ct_date     ON Custody_Transfers(Transfer_Date);
```

PL/SQL trigger bodies, the `sp_custody_transfer` procedure, the `fn_validate_chain_integrity` function and the `pkg_chainguard_reports` package follow in the same master file (Sections 5b–5e). They are reproduced inline in §4.3–§4.5 of this report and shipped verbatim in the repository.

Appendix B · Remaining Complex Queries

B.1 Complete custody chain for one Evidence row (3-table join)

```sql
SELECT e.Evidence_ID, e.Tamper_Seal,
       ct.Transfer_ID, ct.Transfer_Date,
       f.Name AS From_Officer, t.Name AS To_Officer,
       ct.Location, ct.Reason, ct.Status, ct.Is_Sealed
FROM   Evidence e
JOIN   Custody_Transfers ct ON e.Evidence_ID      = ct.Evidence_ID
JOIN   Officers f           ON ct.From_Officer_ID = f.Officer_ID
JOIN   Officers t           ON ct.To_Officer_ID   = t.Officer_ID
WHERE  e.Evidence_ID = 1
ORDER  BY ct.Transfer_Date ASC;
```

B.2 All evidence for a case with current custodian (3-table join)

```sql
SELECT c.FIR_Number, c.Crime_Type,
       e.Evidence_ID, e.Description, e.Category, e.Condition, e.Tamper_Seal,
       o.Name AS Current_Custodian, o.Badge_Number, e.Current_Location
FROM   Cases c JOIN Evidence e ON c.Case_ID              = e.Case_ID
               JOIN Officers o ON e.Current_Custodian_ID = o.Officer_ID
WHERE  c.Case_ID = 1
ORDER  BY e.Evidence_ID;
```

B.3 Officers handling evidence from > 2 cases (correlated subquery)

```sql
SELECT o.Officer_ID, o.Name, o.Badge_Number, o.Rank,
       (SELECT COUNT(DISTINCT e2.Case_ID)
        FROM   Custody_Transfers ct2 JOIN Evidence e2 ON ct2.Evidence_ID = e2.Evidence_ID
        WHERE  ct2.From_Officer_ID = o.Officer_ID
           OR  ct2.To_Officer_ID   = o.Officer_ID) AS Cases_Handled
FROM   Officers o
WHERE  ( SELECT COUNT(DISTINCT e.Case_ID)
         FROM   Custody_Transfers ct JOIN Evidence e ON ct.Evidence_ID = e.Evidence_ID
         WHERE  ct.From_Officer_ID = o.Officer_ID OR ct.To_Officer_ID = o.Officer_ID ) > 2
ORDER  BY Cases_Handled DESC;
```

B.4 Evidence count per case + average lab-analysis duration (LEFT JOIN + GROUP BY)

```sql
SELECT c.Case_ID, c.FIR_Number, c.Crime_Type, c.Status,
       COUNT(DISTINCT e.Evidence_ID)              AS Evidence_Count,
       ROUND(AVG(la.End_Date - la.Start_Date), 1) AS Avg_Analysis_Days,
       COUNT(DISTINCT la.Analysis_ID)             AS Total_Analyses
FROM   Cases c
LEFT   JOIN Evidence    e  ON c.Case_ID     = e.Case_ID
LEFT   JOIN Lab_Analysis la ON e.Evidence_ID = la.Evidence_ID AND la.Status = 'COMPLETED'
GROUP  BY c.Case_ID, c.FIR_Number, c.Crime_Type, c.Status
ORDER  BY Evidence_Count DESC;
```

B.5 Evidence overdue in transit — PENDING transfers older than 2 days

```sql
SELECT e.Evidence_ID, e.Description, e.Tamper_Seal,
       c.FIR_Number, c.Crime_Type,
       ct.Transfer_ID, ct.Transfer_Date,
       ROUND(SYSDATE - ct.Transfer_Date) AS Days_Overdue,
       f.Name AS From_Officer, t.Name AS Assigned_To, t.Phone, ct.Location
FROM   Custody_Transfers ct
JOIN   Evidence e ON ct.Evidence_ID     = e.Evidence_ID
JOIN   Cases    c ON e.Case_ID          = c.Case_ID
JOIN   Officers f ON ct.From_Officer_ID = f.Officer_ID
JOIN   Officers t ON ct.To_Officer_ID   = t.Officer_ID
WHERE  ct.Status = 'PENDING' AND ct.Transfer_Date < SYSDATE - 2
ORDER  BY Days_Overdue DESC;
```

Appendix C · Reproduction Guide

```bash
# 1. Clone the repository
git clone https://github.com/Arun07AK/ChainGuard-DBMS.git
cd ChainGuard-DBMS

# 2. Run the master Oracle script (Live SQL or any 19c+ schema)
#    File: ChainGuard_Complete.sql  -- DDL + DML + triggers + procedures + queries

# 3. Run the Flask dashboard locally (SQLite back-end)
cd chainguard-app
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export CHAINGUARD_DB_PATH=./chainguard.db
python init_db.py        # idempotent; creates tables + seed data
python app.py            # http://localhost:5000

# 4. Production deployment is automatic from main on push (Railway).
#    The same code path runs in CI; no nixpacks edits required.
```

Repository: `github.com/Arun07AK/ChainGuard-DBMS` · Live: `chainguard-dbms-production.up.railway.app`